

Log Platform — Guida all'installazione

Documento generato con Claude

21 agosto 2026

Contents

Panoramica	1
Requisiti	2
Procedura passo-passo	2
1. Copia il pacchetto sul server	2
2. Cifratura del volume dati	2
3. Variabili d'ambiente	2
4. Certificato TLS pubblico (per il dominio)	2
5. CA interna per gli agent (mTLS)	2
6. Avvio dello stack	3
7. Crea l'input GELF TCP+TLS su Graylog	3
8. Primo utente amministratore	3
9. Onboarding di un nuovo tenant/cliente	3
Endpoint Windows	3
Synology (Container Manager) e QNAP (Container Station)	4
10. (Opzionale) Triage AI degli allarmi	4
11. Accesso all'interfaccia	4
12. Backup automatico	4
13. Conservazione a norma dei log amministratori di sistema	5
13.1 Tagga i log di accesso nell'agent	5
13.2 Crea lo stream a conservazione lunga	5
13.3 Sigillo giornaliero (integrità)	5
13.4 Verifica (utile in caso di ispezione)	5
Nota onesta	5
Manutenzione	5
Limiti noti	6

Panoramica

Log Platform è uno stack basato su Graylog per la raccolta e l'analisi dei log di più clienti (multi-tenant), con:

- accesso all'interfaccia protetto da login + MFA (TOTP);
- invio dei log dai client via GELF su TCP con TLS mutuo (mTLS), così solo gli agent autorizzati possono scrivere log;
- separazione dei dati per tenant (stream + index set dedicati per cliente);
- triage automatico degli allarmi tramite Claude (l'IA analizza l'allarme e propone severità reale, causa probabile, azione consigliata).

Questa guida copre l'installazione su un server Linux con Docker. Per NethServer 8 vedi in aggiunta `deploy-ns8/README.md` nel pacchetto.

Requisiti

- Un server Linux (Debian/Ubuntu consigliato) con almeno 4 vCPU e 8 GB di RAM (OpenSearch da solo ne consuma diversi GB).
- Docker Engine e Docker Compose plugin installati.
- Un dominio pubblico che punti al server, per il certificato TLS.
- Un dispositivo/partizione dedicato ai dati, da cifrare (LUKS).
- Una chiave API Anthropic, se si vuole usare il triage AI degli allarmi.

Procedura passo-passo

1. Copia il pacchetto sul server

```
scp -r final-package/ utente@server:/opt/logplatform
ssh utente@server
cd /opt/logplatform
```

2. Cifratura del volume dati

```
sudo ./scripts/setup-encrypted-volume.sh /dev/sdX /mnt/graylog-data
```

Lo script formatta con LUKS il dispositivo indicato e lo monta su `/mnt/graylog-data`. **Attenzione:** cancella tutti i dati presenti sul device. Segui a schermo le istruzioni per rendere il mount persistente al riavvio (richiede una passphrase o una keyfile protetta — vedi output dello script).

3. Variabili d'ambiente

```
cp .env.example .env
nano .env
```

Compila **tutte** le voci segnate **CAMBIAMI**: password di MariaDB, il segreto di Graylog, la chiave API Anthropic, il dominio pubblico, ecc. Genera stringhe casuali robuste, ad esempio:

```
openssl rand -hex 32
```

4. Certificato TLS pubblico (per il dominio)

Metti `fullchain.pem` e `privkey.pem` in `nginx/certs/`, ad esempio ottenuti con `certbot`:

```
sudo certbot certonly --standalone -d logs.tuodominio.it
cp /etc/letsencrypt/live/logs.tuodominio.it/fullchain.pem nginx/certs/
cp /etc/letsencrypt/live/logs.tuodominio.it/privkey.pem nginx/certs/
```

5. CA interna per gli agent (mTLS)

```
./scripts/generate-ca.sh
```

Genera la CA interna e il certificato server per l'input GELF di Graylog. **Conserva** `ca/ca-key.pem` con cura: chiunque la possieda può emettere certificati validi per inviare log alla piattaforma.

6. Avvio dello stack

```
docker compose up -d
docker compose ps    # verifica che tutti i 7 servizi siano "running"
```

Al primo avvio Graylog e OpenSearch impiegano qualche minuto per inicializzarsi.

7. Crea l'input GELF TCP+TLS su Graylog

```
./scripts/create-gelf-tls-input.sh
```

Ti chiederà la password admin di Graylog (quella impostata via `GRAYLOG_ROOT_PASSWORD_SHA2` in `.env`, in chiaro qui perché serve per la chiamata API — non per il file).

8. Primo utente amministratore

```
docker compose exec auth-service python -m app.create_user
```

Segui le domande a schermo (username, password, tenant vuoto per l'accesso da amministratore globale).

9. Onboarding di un nuovo tenant/cliente

```
./scripts/provision-tenant.sh acme
```

Annota lo **Stream ID** e l'**Index set ID** stampati in output.

```
./scripts/generate-agent.sh acme logs.tuodominio.it
```

Produce `dist/agent-acme.tar.gz`: contiene `agent.py`, `agent.ini` (da rifinire con hostname e percorsi dei file di log reali), il certificato client e la CA. Da installare sull'endpoint del cliente:

```
tar xzf agent-acme.tar.gz
cd agent-acme
nano agent.ini    # imposta hostname e log_files
python3 agent.py --config agent.ini
```

Valuta di eseguirlo come servizio systemd persistente sull'endpoint del cliente. Nel pacchetto trovi `logplatform-agent.service.example`:

```
sudo cp logplatform-agent.service.example /etc/systemd/system/logplatform-agent.service
sudo nano /etc/systemd/system/logplatform-agent.service    # adatta i percorsi
sudo systemctl daemon-reload
sudo systemctl enable --now logplatform-agent
```

Endpoint Windows

Il pacchetto generato da `generate-agent.sh` include anche `agent_windows.py`, pensato per girare come **servizio Windows nativo** (nessun tool esterno richiesto) e capace di leggere sia file di log testuali (es. log IIS) sia canali dell'Event Log di Windows (Application, System, ecc.).

Da un prompt **Amministratore**, nella cartella del pacchetto estratto:

```
pip install -r requirements-windows.txt
notepad agent.ini    # imposta hostname e, se vuoi, event_logs = Application, System
python agent_windows.py debug    # test in primo piano, Ctrl+C per fermare
python agent_windows.py install
python agent_windows.py start
```

Comandi utili: `stop`, `remove`, `restart`. Lo stato di lettura dell'Event Log (ultimo record inviato) viene salvato in `agent_state.json` accanto al config, così un riavvio del servizio non rimanda tutto da capo.

Synology (Container Manager) e QNAP (Container Station)

Per questi NAS l'agent gira **containerizzato** (Docker), riusando lo stesso `agent.py` Linux: sia DSM 7+ (Container Manager) sia QTS/QuTS hero (Container Station) supportano Docker/Compose nativamente sui modelli x86. Non esiste invece, in questo pacchetto, un pacchetto nativo SPK (Synology) o QPKG (QNAP): richiederebbero i toolchain ufficiali dei rispettivi vendor e non sono stati validati.

1. Copia la cartella `agent-<tenant>/` generata da `generate-agent.sh` sul NAS (File Station o una cartella condivisa qualsiasi).
2. Rinomina `config-nas/` in `config/` (contiene già un `agent.ini` con i percorsi corretti per l'esecuzione in container, cioè `/config/...` e `/logs/...`).
3. Apri `docker-compose.nas.yml.example`, adatta i volumi ai percorsi di log reali del tuo NAS (i log di sistema variano tra Synology e QNAP: verifica in Log Center / File Station quali cartelle vuoi monitorare), poi rinominalo in `docker-compose.yml`.
4. **Synology:** Container Manager → scheda “Progetto” → Crea → incolla il contenuto di `docker-compose.yml` → Avvia. **QNAP:** Container Station → Crea → “Crea applicazione” → “Crea da YAML” → incolla il contenuto → Crea.
5. Verifica dai log del container (Container Manager/Station hanno una vista log) che l'agent si sia connesso senza errori TLS, poi controlla su Graylog che i messaggi arrivino nello stream del tenant.

L'agent effettua sempre connessioni in **uscita** verso Graylog: non serve aprire porte in ingresso sul NAS.

10. (Opzionale) Triage AI degli allarmi

```
cp ai-service/tenant_webhooks.json.example ai-service/tenant_webhooks.json
nano ai-service/tenant_webhooks.json    # webhook Slack/Teams del tenant, opzionale
docker compose restart ai-service

./scripts/create-alert-notification.sh acme <STREAM_ID_del_tenant>
```

Questo crea un esempio di condizione di allarme (soglia messaggi) collegata al triage AI. Vai nella UI Graylog (Alerts → Event Definitions) per rifinire la condizione secondo le tue esigenze reali.

11. Accesso all'interfaccia

Apri <https://logs.tuodominio.it/login.html>, inserisci utente e password, poi scansiona il QR con un'app authenticator (Google Authenticator, Authy, ecc.) e inserisci il codice a 6 cifre. Dai login successivi basterà inserire il codice generato dall'app.

12. Backup automatico

```
crontab -e
```

Aggiungi:

```
0 3 * * * /opt/logplatform/scripts/backup.sh >> /var/log/logplatform-backup.log 2>&1
*/5 * * * * /opt/logplatform/scripts/watchdog.sh >> /var/log/logplatform-watchdog.log 2>&1
```

Il backup copre MariaDB (utenti/MFA), la configurazione Graylog (MongoDB), la CA e i certificati. **Non include gli indici OpenSearch** (i log veri e propri) per motivi di spazio: se ti servono, valuta snapshot OpenSearch dedicati.

13. Conservazione a norma dei log amministratori di sistema

Se hai bisogno di essere conforme al **Provvedimento Garante Privacy 27/11/2008** sugli Amministratori di Sistema (conservazione minima 6 mesi, completezza, non alterabilità dei log di accesso), la piattaforma include un flusso dedicato, separato dai log generici del tenant.

13.1 Tagga i log di accesso nell'agent

Negli endpoint del tenant, in `agent.ini`, sposta i file/canali che contengono accessi amministrativi (login, sudo, logon Windows) da `log_files/event_logs` a `admin_log_files/admin_event_logs`:

```
admin_log_files = /var/log/auth.log
# Windows:
# admin_event_logs = Security
```

Questi vengono taggati automaticamente `log_class=admin-access`.

13.2 Crea lo stream a conservazione lunga

```
./scripts/provision-admin-log-stream.sh acme 200
```

Crea uno stream + index set separati con retention di 200 giorni (oltre i 6 mesi minimi) per i soli log con `tenant=acme` AND `log_class=admin-access`.

13.3 Sigillo giornaliero (integrità)

La retention da sola copre solo la *durata*, non la *non alterabilità*. Aggiungi a cron:

```
crontab -e
```

```
15 1 * * * /opt/logplatform/scripts/seal-admin-logs.sh acme >> /var/log/logplatform-seal.log 2>&1
```

Ogni notte lo script: 1. esporta i log del giorno prima dallo stream `admin-access` del tenant; 2. calcola uno SHA-256 e lo concatena a quello del giorno precedente (hash chain: un'alterazione a posteriori rompe la catena e si vede); 3. se raggiungibile, chiede una marca temporale RFC 3161 a una TSA esterna (di default freetsa.org, gratuita e best-effort — per un valore probatorio più solido valuta una TSA qualificata a pagamento); 4. rende i file immutabili a livello filesystem (`chattr +i`, se supportato — **non è WORM hardware**: un root compromesso potrebbe rimuoverlo, la protezione reale viene dalla combinazione con la marca temporale esterna).

13.4 Verifica (utile in caso di ispezione)

```
./scripts/verify-admin-log-seal.sh acme
```

Ricalcola l'intera catena e segnala qualunque file mancante, alterato, o con una marca temporale non più verificabile.

Nota onesta

Questo meccanismo copre in modo ragionevole i tre requisiti della normativa (durata, completezza monitorabile, integrità verificabile), ma non è una certificazione: in caso di contestazione legale, il valore probatorio dipende anche dalla qualità della TSA usata e dalla robustezza dei controlli di accesso alla macchina che ospita i sigilli. Per contesti ad alto rischio, valuta un consulente legale/DPO e un servizio di conservazione sostitutiva accreditato AgID per i casi che lo richiedono.

Manutenzione

Attività	Comando
Vedere lo stato dei servizi	<code>docker compose ps</code>
Vedere i log di un servizio	<code>docker compose logs -f graylog</code>
Riavviare un servizio	<code>docker compose restart <servizio></code>
Ripristinare un backup	<code>./scripts/restore.sh</code> <code>/percorso/backup/<timestamp></code>
Aggiungere un nuovo utente	<code>docker compose exec auth-service python -m</code> <code>app.create_user</code>

Limiti noti

- Lo stack è a singolo nodo: non c'è alta affidabilità/failover automatico.
- Il modulo per NethServer 8 (`ns8-logplatform/`) è uno scaffold non ancora validato su un cluster reale — vedi `deploy-ns8/README.md`.
- Il triage AI è un supporto decisionale, non sostituisce l'analisi umana per allarmi critici.